

UNIT-II

CONVOLUTION NEURAL NETWORK (CNN): Introduction to CNNs and their applications in computer vision, CNN basic architecture, Activation functions- sigmoid, tanh, ReLU, Leaky ReLU, Softmax layer, Types of pooling layers, Training of CNN in TensorFlow, various popular CNN architectures: VGG, Google Net, ResNet etc, Dropout, Normalization, Data augmentation.

Introduction to CNNs and Their Applications in Computer Vision

Convolutional Neural Networks (CNNs) are a specialized class of deep learning models designed primarily for processing data with a grid-like topology, such as images. They have revolutionized the field of computer vision by achieving remarkable accuracy in tasks like image classification, object detection, segmentation, and facial recognition. CNNs are inspired by the structure of the visual cortex in animals, where neurons respond to overlapping regions of the visual field. This biological motivation helps CNNs recognize patterns and features in images effectively.

The key idea behind CNNs is the convolution operation, which replaces traditional matrix multiplication in at least one of the layers of a neural network. This operation uses small filters or kernels to scan through the input image and extract local features such as edges, textures, or colors. These features are then passed through layers of increasing depth, where more abstract patterns are recognized, such as shapes or entire objects.

One of the main advantages of CNNs is their parameter efficiency. Unlike fully connected networks that treat all pixels equally and require a large number of weights, CNNs share weights through filters, significantly reducing the number of parameters. This makes CNNs more computationally efficient and less prone to overfitting, especially on high-dimensional input like images.

CNNs are structured hierarchically, with early layers capturing low-level features (edges, corners), and deeper layers recognizing high-level semantic features (like eyes, wheels, or digits). This layered feature learning process allows CNNs to automatically extract features without the need for manual feature engineering.

In terms of applications, CNNs have become the backbone of modern computer vision systems. In image classification, CNNs can distinguish between thousands of categories such as cats, dogs, and airplanes. In object detection, they not only classify objects but also localize them using bounding boxes. Semantic segmentation uses CNNs to label each pixel in an image with a class label. In medical imaging, CNNs are used to identify tumors in X-rays, MRIs, or CT scans. They are also used in autonomous vehicles to detect pedestrians, lane markings, and traffic signs.

Further applications include facial recognition, optical character recognition (OCR), gesture recognition, and style transfer. CNNs also play a critical role in content-based image retrieval systems and are increasingly being used in video analysis, such as action recognition and video classification.

CNNs provide a powerful framework for handling image and video data by leveraging spatial hierarchies and local feature extraction. Their success in real-world applications continues to grow, making them a fundamental component of modern artificial intelligence systems.

CNN basic architecture

The basic architecture of a Convolutional Neural Network (CNN) is composed of a sequence of layers, each of which transforms the input data into increasingly abstract and informative representations. The architecture typically includes three main types of layers: convolutional layers, pooling layers, and fully connected layers. Each of these plays a distinct role in processing the input data, particularly images.

The input layer of a CNN receives raw pixel values from an image. For example, a grayscale image of size 28×28 is represented as a matrix of pixel intensity values, while a color image has three channels (Red, Green, Blue), forming a 3D tensor.

The convolutional layer is the core building block of a CNN. It uses small learnable filters (e.g., 3×3 or 5×5) that are convolved over the input to produce feature maps. Each filter is trained to detect a specific feature such as vertical edges, horizontal edges, or patterns. The convolution operation preserves the spatial relationship between pixels by learning local patterns, which makes CNNs particularly powerful in visual tasks.

After the convolution operation, the output is passed through a non-linear activation function like ReLU (Rectified Linear Unit), which introduces non-linearity into the model, allowing it to learn complex mappings.

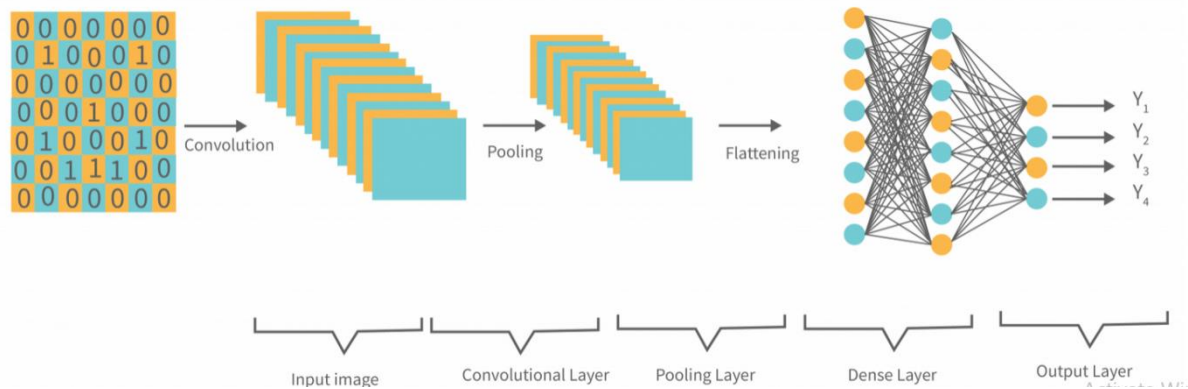
Next comes the pooling layer, which performs downsampling. Pooling reduces the spatial dimensions (width and height) of the feature maps while retaining the most important information. This not only reduces computational cost but also provides translation invariance. Common pooling techniques include max pooling and average pooling.

The flattening layer converts the 2D feature maps into a 1D vector, which is then fed into one or more fully connected (dense) layers. These layers are similar to those in traditional neural networks and are used to combine the features learned by the convolutional and pooling layers to make final predictions.

Finally, the output layer produces the prediction results. For classification tasks, it often uses a softmax activation function, which converts the output into a probability distribution across different classes.

A CNN architecture may also include regularization techniques such as dropout to prevent overfitting and batch normalization to stabilize training. The design of the architecture (number of layers, filter sizes, stride, padding, etc.) is usually tailored to the specific problem and dataset.

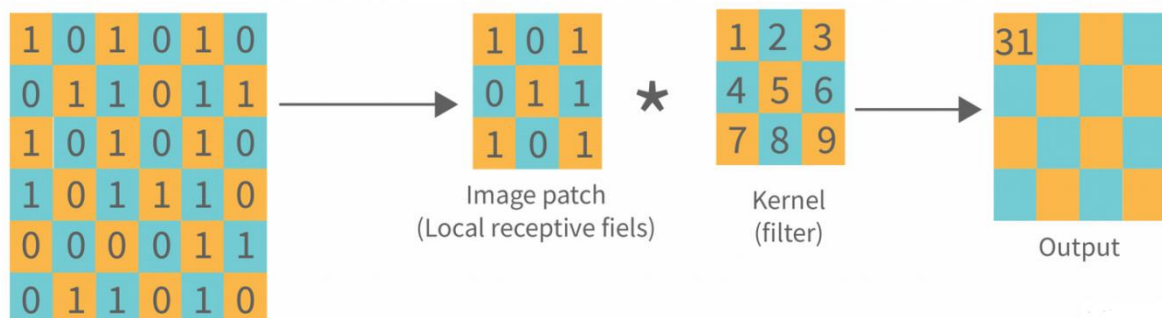
To summarize, the basic architecture of a CNN transforms raw image input through a series of convolution, activation, and pooling layers into a meaningful output. This architecture allows the model to learn hierarchical representations of data — from low-level features like edges to high-level semantics like objects — making CNNs the foundation of modern computer vision solutions.



The ConvNet's job is to compress the images into a format that is easier to process while preserving elements that are important for obtaining a decent prediction. This is critical for designing an architecture that is capable of learning features while also being scalable to large datasets.

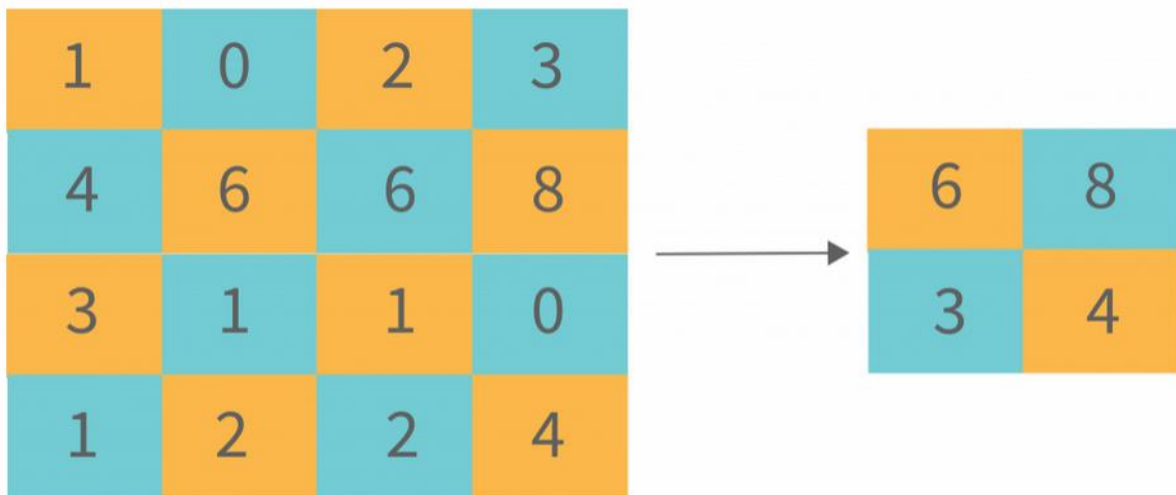
A convolutional neural network, ConvNets in short has three layers which are its building blocks, let's have a look:

Convolutional Layer (CONV): They are the foundation of CNN, and they are in charge of executing convolution operations. The Kernel/Filter is the component in this layer that performs the convolution operation (matrix). Until the complete image is scanned, the kernel makes horizontal and vertical adjustments dependent on the stride rate. The kernel is less in size than a picture, but it has more depth. This means that if the image has three (RGB) channels, the kernel height and width will be modest spatially, but the depth will span all three.

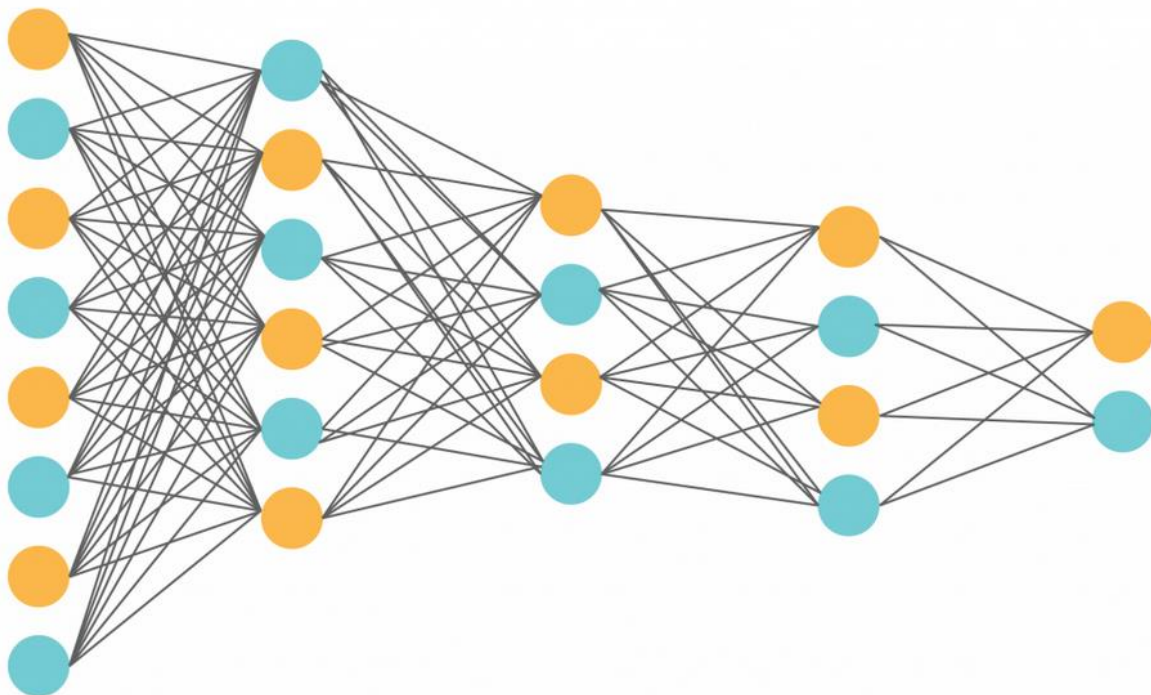


Other than convolution, there is another important part of convolutional layers, known as the Non-linear activation function. The outputs of the linear operations like convolution are passed through a non-linear activation function. Although smooth nonlinear functions such as the sigmoid or hyperbolic tangent (tanh) function were formerly utilized because they are mathematical representations of biological neuron actions. The rectified linear unit (ReLU) is now the most commonly used non-linear activation function. $f(x) = \max(0, x)$

Pooling Layer (POOL): This layer is in charge of reducing dimensionality. It aids in reducing the amount of computing power required to process the data. Pooling can be divided into two types: maximum pooling and average pooling. The maximum value from the area covered by the kernel on the image is returned by max pooling. The average of all the values in the part of the image covered by the kernel is returned by average pooling.



Fully Connected Layer (FC): The fully connected layer (FC) works with a flattened input, which means that each input is coupled to every neuron. After that, the flattened vector is sent via a few additional FC layers, where the mathematical functional operations are normally performed. The classification procedure gets started at this point. FC layers are frequently found near the end of CNN architectures if they are present.



Along with the above layers, there are some additional terms that are part of a CNN architecture.

Activation Function: The last fully connected layer's activation function is frequently distinct from the others. Each activity necessitates the selection of an appropriate activation function. The softmax function, which normalizes output real values from the last fully connected layer to target class probabilities, where each value ranges between 0 and 1 and all values total to 1, is an activation function used in the multiclass classification problem.

Dropout Layers: The Dropout layer is a mask that nullifies some neurons' contributions to the following layer while leaving all others unchanged. A Dropout layer can be applied to the input

vector, nullifying some of its properties; however, it can also be applied to a hidden layer, nullifying some hidden neurons. Dropout layers are critical in CNN training because they prevent the training data from overfitting. If they aren't there, the first batch of training data has a disproportionately large impact on learning. As a result, learning of traits that occur only in later samples or batches would be prevented.

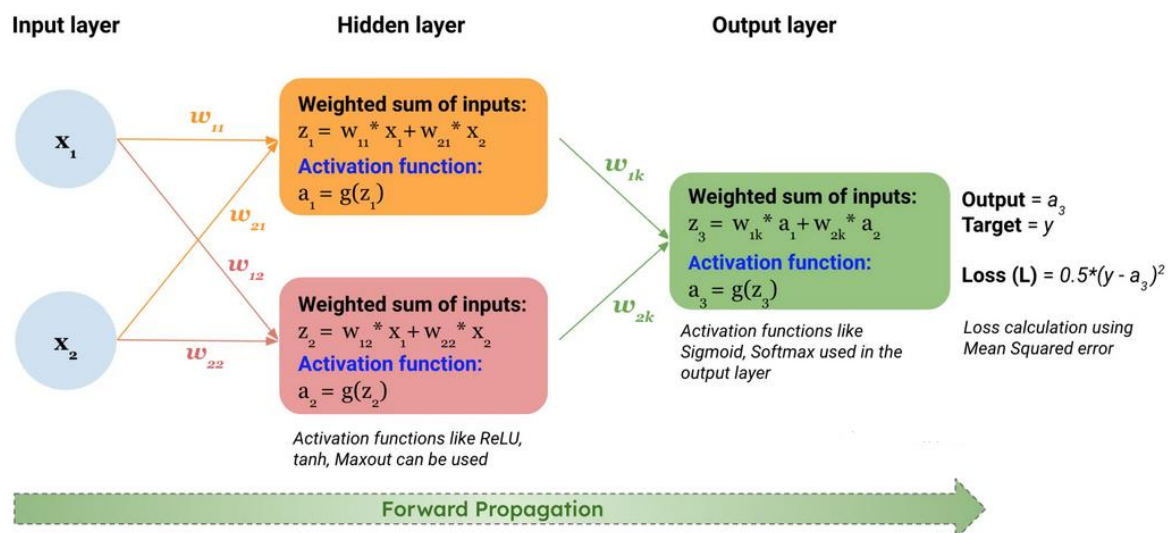
Advantages of CNN Architecture

Following are some of the advantages of a Convolutional Neural Network:

- CNN is computationally efficient.
- It performs parameter sharing and uses special convolution and pooling algorithms. CNN models may now run on any device, making them globally appealing.
- It finds the relevant features without the need for human intervention.
- It can be utilized in a variety of industries to execute key tasks such as facial recognition, document analysis, climate comprehension, image recognition, and item identification, among others.
- By feeding your data on each level and tuning the CNN a little for a specific purpose, you can extract valuable features from an already trained CNN with its taught weights.
-

Activation functions- sigmoid, tanh, ReLU, Leaky ReLU, Softmax layer

Activation functions are a critical component of neural networks, including CNNs. They determine whether a neuron should be activated or not by introducing non-linearity into the model. Without activation functions, the network would behave like a linear regression model, unable to capture complex patterns.



Activation functions serve two main purposes:

1. **Introduce Non-linearity:** Without non-linearity, a neural network would behave like a linear model, no matter how deep it is. Activation functions allow the network to learn and represent complex, non-linear mappings between inputs and outputs.
2. **Control Neuron Activation:** Activation functions control the firing behavior of neurons. Depending on the activation function's output, a neuron might become activated (output a non-zero value) or remain inactive (output zero).

Typically, same activation function is applied to all the hidden layers, while the output layer uses a different activation function, based on the type of prediction model aims to make.

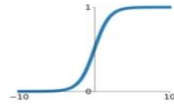
Different types of Activation Functions:

Activation functions, which are popularly used in neural network models, are shown in the figure below.

Activation Functions

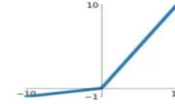
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



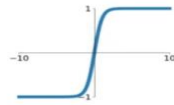
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

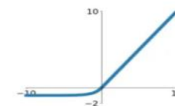
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



The sigmoid function is defined as $\sigma(x) = \frac{1}{1+e^{-x}}$.

It maps input values into the range (0, 1), making it useful for binary classification problems. However, it suffers from the vanishing gradient problem, where gradients become too small for large or small input values, slowing down or stopping learning.

The tanh (hyperbolic tangent) function is given by $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$.

It maps values to the range (-1, 1), and like sigmoid, it has a smooth gradient, but with a zero-centered output, which helps in optimization. Still, it suffers from vanishing gradients for large input values.

The ReLU (Rectified Linear Unit) function is defined as $f(x) = \max(0, x)$.

It has become the most commonly used activation in CNNs due to its simplicity and effectiveness. It allows the network to converge faster and avoids the vanishing gradient issue. However, ReLU can suffer from the “dying ReLU” problem, where neurons get stuck during training and output zero for all inputs.

To address this, Leaky ReLU introduces a small slope for negative input values, defined as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where α is a small constant like 0.01. This modification allows negative input values to propagate and prevents neurons from dying.

For classification tasks, especially multi-class, the Softmax function is used in the final layer. It converts a vector of raw scores into probabilities by emphasizing the highest values and suppressing

the others. Each output is in the range (0,1) and sums to 1 across all classes, making it ideal for deciding the most likely class.

Pros and Cons of different Activation Functions

Used in layer	Activation Function	Details	Pros	Cons
Hidden / Output	Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$ Output range: [0,1]	- Smooth activation that outputs values between 0 and 1, making it suitable for binary classification - Historically popular	- Vanishing gradient problem due to saturated neurons - Output not zero-centered as sigmoid outputs are always positive - exp() operation is computationally intensive
Hidden	Tanh (Hyperbolic tangent)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Output range: [-1,1]	- Zero centered outputs that help networks train faster	- Suffers with vanishing gradient problem when saturated
Hidden	ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$ Output range: [0, ∞)	- Does not saturate: avoids vanishing gradient issues for positive inputs - Computationally efficient since only certain number of neurons are activated at the same time - Much faster convergence compared to sigmoid/tanh	- Output not zero-centered - Prone to a "dying ReLU" problem, where neurons can get stuck during training and never activate again, leading to a dead neuron that doesn't update its weights.

Used in layer	Activation Function	Details	Pros	Cons
Hidden	Leaky ReLU	$f(x) = \max(0.01x, x)$ Output range: $(-\infty, \infty)$	- All benefits of ReLU - Addresses the "dying ReLU" problem by allowing a small gradient for negative inputs. Helps with training deeper networks	- Not as standardized as ReLU, and the slope of the leaky part is typically a hyperparameter (α) that needs tuning. This is also referred to as Parametric ReLU $f(x) = \max(\alpha x, x)$
Hidden	ELU (Exponential Linear Unit)	where $\alpha > 0$: $f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$ α is commonly chosen as 1 Output range: $(-\alpha, \infty)$	- All benefits of ReLU - Zero centered outputs that help networks train faster - ELUs saturate to a negative value when the argument gets smaller becoming more robust to noise	- Computationally more expensive due to exponential operation

Activation functions are essential for learning complex, non-linear relationships in data. Choosing the right activation function affects the learning efficiency, convergence speed, and overall performance of CNN models.

Types of pooling layers

Pooling layers are a fundamental part of Convolutional Neural Networks (CNNs). Their main function is to progressively reduce the spatial dimensions (width and height) of the feature maps, which helps in reducing the number of parameters and computations in the network. Pooling also provides translation invariance, meaning that small shifts in the input image do not significantly change the output of the model.

Pooling operates independently on every depth slice (channel) of the input and resizes it spatially. The most commonly used types of pooling are max pooling, average pooling, and global pooling.

Max Pooling is the most widely used form of pooling. In this method, a filter (e.g., 2×2) slides over the input feature map, and for each region, the maximum value is taken. This effectively captures the most prominent feature (strongest activation) in that region. Max pooling retains sharp and distinct features and is commonly used after convolution and activation layers.

For example, given a 2×2 region:

$$\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$$

Max pooling outputs: 4

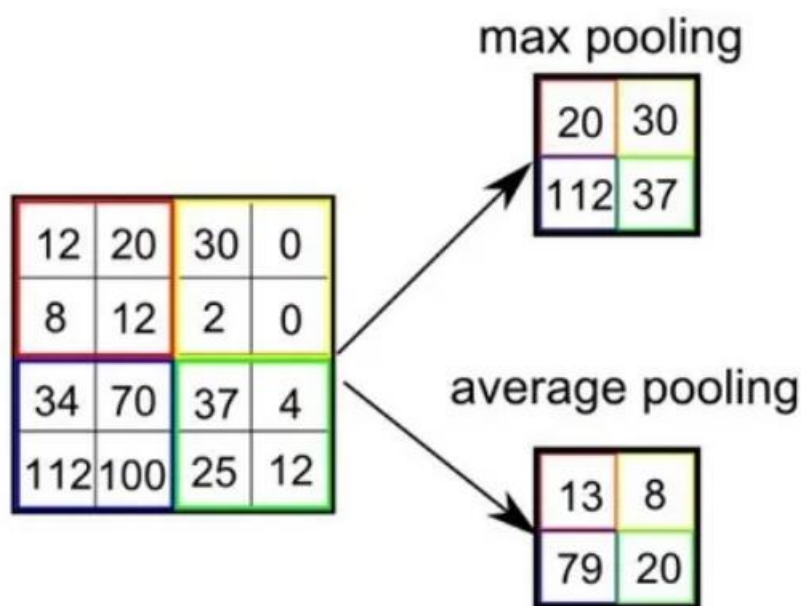
Average Pooling computes the average of the values in each region covered by the filter. It is less aggressive than max pooling and provides smoother representations. Average pooling may be useful in certain applications where subtle patterns and overall trends are more important than the strongest signals.

- Max Pooling- calculates maximum of the each block of feature map.
- Average Pooling- calculates average of the each block of feature map.
- Min Pooling- calculates minimum of the each block of feature map.

Why Max Pooling is used?

Two main reasons why max pooling is effective is :

- It reduces the amount of parameters going forward and hence computational load.
- Higher valued pixels are the most activated and hence captured in this operation.



Max pooling and Average Pooling comparison

Max Pooling Process

Since we have got a basic idea what max pooling intends to do, let's discuss about the operation what it actually does.

We have discussed about role of kernels in the previous post. They act as filters and when they convolve over an image, they create an output corresponding to computations of the filter applied.

For max pooling, we define

- filter(or kernel) of size $n \times n$
- stride value k (by how many pixels we want our filter to move),

For each movement of the filter from the $n \times n$ pixels block of the image under consideration at that point, maximum value is captured as the output for the next layer. Then the filter is moved by k pixels (defined as stride) to perform the same operation again. One key point to be noted that Max pooling is applied after a convolutional layer.



Max pooling using kernel of size 2×2 with stride 2

Global Pooling is a special case of pooling where the pooling operation is applied over the entire feature map instead of small regions. In global max pooling, the maximum value from the entire map is chosen, and in global average pooling, the average of all values is computed. These are often used before the final output layer to convert feature maps into a single vector, especially in architectures designed for image classification.

In addition to the type of pooling, the choice of stride (the number of pixels the filter moves) and whether or not to use padding (adding zeros around the input) also affects the output size and information retained. A stride of 2 is common and reduces the size of feature maps by half in both dimensions.

While pooling is extremely useful, it also leads to loss of spatial information. In recent architectures, alternatives like strided convolutions and attention mechanisms are being explored to retain more detailed spatial information.

In conclusion, pooling is an essential technique in CNNs for dimensionality reduction, improved generalization, and computational efficiency. The choice between max, average, or global pooling depends on the specific task and the desired behavior of the model.

Training of CNN in TensorFlow

Training a Convolutional Neural Network (CNN) using TensorFlow, an open-source deep learning framework developed by Google, is one of the most efficient and scalable ways to build and deploy